2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT) | 979-8-3503-3509-5/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICCCNT56998.2023.10306457

Advancing Multimodal Fusion in Human-Computer Interaction: Integrating Eye Tracking, Lips Detection, Speech Recognition, and Voice Synthesis for Intelligent Cursor Control and Auditory Feedback

1st Gayatri Jagnade
Department of Multidisciplinary Engineering
Vishwakarma Institute of Technology
Pune, India
gayatri.jagnade@vit.edu

2nd Sunil Sable
Department of Chemical Engineering
Vishwakarma Institute of Technology
Pune, India
sunil.sable@vit.edu

3rd Mitesh Ikar
Department Chemical Engineering
Vishwakarma Institute of Technology
Pune, India
mitesh.ikar21@vit.edu

Abstract- **In the realm of human-computer interaction (HCI), the integration of multiple modalities has emerged as a promising approach to enhance user experiences and enable more natural and intuitive interaction paradigms. This paper explores the advancements in multimodal fusion techniques and their application in HCI, focusing on the integration of eye tracking, lips detection, speech recognition, and voice synthesis. The proposed framework aims to leverage the combined power of these modalities to achieve intelligent cursor control and provide enhanced auditory feedback to users. By seamlessly integrating eye tracking, lips detection, and speech recognition, the system can accurately interpret user intentions and gestures. The eye tracking component enables precise tracking of gaze movements, while the lips detection module analyses lip movements for additional input signals. Concurrently, the speech recognition module captures user commands and converts them into actionable instructions. The synthesized voice feedback component plays a crucial role in providing auditory responses to user actions, ensuring a seamless and engaging interaction experience. Through the integration of voice synthesis, the system can effectively communicate feedback, confirm user inputs, and guide users through the interface. The combined functionality of intelligent cursor control and auditory feedback opens up new possibilities for more efficient and immersive HCI. Experimental evaluations demonstrate the effectiveness of the proposed multimodal fusion framework. User studies reveal improved cursor manipulation accuracy and reduced cognitive load compared to traditional input methods. The system's ability to adapt to individual user preferences and environmental conditions further enhances the overall user experience. This paper contributes to the advancement of multimodal fusion techniques in HCI by showcasing the integration of eye tracking, lips detection, speech recognition, and voice synthesis. It highlights the potential of this comprehensive approach to enable intelligent cursor control and provide sophisticated auditory feedback, thereby paving the way for more intuitive and immersive human-computer interaction paradigms.**

Keywords- Auditory Feedback; Eye Tracking; Human-Computer Interaction; Multimodal Fusion; Speech Recognition

I. INTRODUCTION

Human-Computer Interaction (HCI) plays a pivotal role in shaping the way individuals interact with digital systems and technologies. With the aim of creating more intuitive and seamless user experiences, researchers and developers have been exploring novel approaches that integrate multiple modalities to enhance interaction paradigms. This paper focuses on advancing multimodal fusion techniques in HCI by integrating eye tracking, speech recognition, and auditory feedback. Multimodal fusion refers to the integration and combination of data from different modalities, such as vision and speech, to extract more robust and meaningful information. By harnessing the complementary nature of different modalities, a more comprehensive understanding of user inputs and intentions can be achieved, leading to improved interaction and usability. Eye tracking is a powerful technology that allows for the precise tracking of users' gaze movements. It provides valuable insights into users' visual attention, aiding in understanding their focus of interest and intention during interactions with digital interfaces. Speech recognition, on the other hand, enables the conversion of spoken commands and instructions into actionable input. This allows for a more natural and hands-free interaction with digital systems. In this paper, we explore the integration of eye tracking and speech recognition in the context of HCI. By combining these modalities, we aim to enable intelligent cursor control and provide auditory feedback to enhance the user experience. The eye tracking component enables precise tracking of the user's gaze, while the speech recognition module captures spoken commands and converts them into actionable instructions. The fusion of these modalities offers a more comprehensive understanding of user input, enabling more precise and context-aware interaction. Furthermore, we introduce auditory feedback as a means to provide real-time responses and guidance to users. By employing voice synthesis techniques, the system can deliver auditory feedback that confirms user inputs, provides informative cues, and guides users through the interface. This auditory feedback adds an additional layer of engagement and assistance, making the interaction more

informative and immersive. In this paper, we explore the integration of eye tracking and speech recognition within the context of HCI. By combining these modalities, our aim is to enable intelligent cursor control and provide auditory feedback to enhance the user experience. The eye tracking component allows for precise tracking of the user's gaze, while the speech recognition module captures spoken commands and converts them into actionable instructions. The fusion of these modalities offers a more comprehensive understanding of user input, enabling more precise and context-aware interaction. Furthermore, we introduce auditory feedback as a means to provide real-time responses and guidance to users. By employing voice synthesis techniques, our system delivers auditory feedback that confirms user inputs, provides informative cues, and guides users through the interface. This auditory feedback adds an additional layer of engagement and assistance, making the interaction more informative and immersive. Through the integration of eye tracking, speech recognition, and auditory feedback, our goal is to push the boundaries of multimodal fusion in HCI. By leveraging the strengths of these modalities, we can create more intelligent and responsive systems that enhance user interaction and improve overall usability. The subsequent sections of this paper will delve into the technical details of the proposed framework, present experimental evaluations, and discuss the implications and potential future directions of this research. Through the integration of eye tracking, speech recognition, and auditory feedback, we aim to advance the state-of-the-art in multimodal fusion for HCI. By leveraging the strengths of these modalities, we can create more intelligent and responsive systems that enhance user interaction and usability. The subsequent sections of this paper will delve into the technical details of the proposed framework, present experimental evaluations, and discuss the implications and potential future directions of this research.

II. LITERATURE REVIEW

Numerous researchers have made significant contributions to the field of eye tracking across various domains, employing different systems and technologies to track users' eye movements. The author focused on addressing the system's slow response to eye detection. They used a Pentium 11333 Mhz processor for pupil and face detection, capturing 30 frames per second. To synchronize the frames, a hardware component that illuminates the pupil was employed as the PCI frame grabber used lacked image processing capabilities [1]. Another approach involves real-time face detection, where each scale and location within each image is analyzed to achieve real-time speed. Some devices utilize color filtering or motion filtering in video clips, but accessing color and movement in extensive databases can be challenging. Vision-based eye tracking plays a crucial role in understanding eye movements and gaze patterns. It enables effective interaction between users and computer systems, providing context about where the user is looking. This technology is widely used for measuring visual attention and facilitating various types of user interaction tasks [2]. One system described in the literature utilizes robust eye-tracking data from multiple cameras to estimate 3D head orientation through triangulation. Multiple cameras offer a larger tracking volume compared to individual sensors. The accuracy of the system's head pose calculation subsystem is validated by comparing the gaze estimates generated by users with those estimated by the system [3]. The author employed OpenCV for real-time video processing on Android smartphones. Two different methods were developed using two distinct approaches: one utilizing the OpenCV library and the other implementing a self-made algorithm called 'CamTest.' Eight image recognition techniques were tested to evaluate the efficiency of the Android library and OpenCV in processing each frame of captured video on an Android smartphone. The study concluded that most image processing techniques using the OpenCV library outperformed the self-made Android library algorithm in terms of performance, with OpenCV emphasizing improvement over energy consumption [4]. Different approaches have been explored for eye detection.The regression approach minimizes the distance between predicted and actual eye positions, the Bayesian approach learns models of eye and non-eye appearance, and the discriminative approach treats the problem as a classification task [5] . In [6] and [7], computer graphics and OpenCV were employed for fatigue detection using smartphones and facial behavior detection using machine learning approaches. These studies contribute to addressing the issue of driver fatigue by developing working prototypes for real-time fatigue detection using Android smartphones or iPhones. The smartphone-based fatigue detection systems monitor visual indicators of driver fatigue, including head gestures, head rotations, and eye blinks. This technology offers a portable and cost-effective alternative to existing fatigue detection systems, potentially improving driving safety. Additionally, computer vision technologies, including machine learning algorithms, play a vital role in enhancing driving safety through various safety techniques such as drowsiness detection, forward stability control, and automatic braking alarms. Several experiments and studies were conducted to evaluate the effectiveness of smartphone-based driver fatigue detection, including a virtual driving experiment to identify visual signs of driver fatigue. Participants underwent screening to ensure eligibility criteria, such as a minimum of two years of driving experience, regular or corrected vision, and the absence of any conditions, intoxication, or medication that could affect the results. The collected data included head movements, eye blinks, and head rotations, which were analyzed as visual behaviors [6,7]. The introduction of a voice recognition system involves the process of converting human voice into digital data. In the context of Tamil language recognition, the author focuses on developing such a system. One of the features utilized in this system is Mel-frequency spectral coefficients which are commonly used for speech analysis [8]. To convert voice into text, various computer functions are employed. These functions involve analyzing the acoustic signal and interpreting the processed signal. The author highlights the importance of two components

in this process. Additionally, technologies like Hidden Markov Model (HMM) are utilized to develop the voice recognition system [9]. HMM is a statistical model commonly used in speech recognition. The author's main emphasis lies on the human-computer interface using automatic speech recognition (ASR) system features. Different methods of ASR techniques are explained in detail [10]. Furthermore, the author discusses Automatic Speech Emotion Recognition, where a system is developed using support vector machines to classify between male and female voices and recognize different human emotions [11]. The paper also touches upon optimization techniques and their applications. It provides a survey of algorithms and applications related to optimization [12]. Additionally, the author addresses challenges faced in speech recognition, such as crowd noise, different accents, and surrounding disturbances. To tackle these issues, the author proposes a speech noise removal technique employing various processing methods [13].

While reviewing the existing literature, it was observed that the technologies used often require optimal lighting conditions, and the performance measure tends to decline when using RGB image formats. Furthermore, the systems discussed in the literature rely on high pixel rates for cameras to accurately track users' faces. To overcome these limitations, a proposed system for detecting and monitoring facial gestures using the dlib library is presented. The system processes real-time video input, performs face recognition, and tracks facial movements, allowing users to navigate and control various motions on the screen through facial gestures. The input images are converted into grayscale images, enabling efficient tracking of facial movements and facilitating cursor movement based on the detected gestures. The reviewed literature highlights the contributions of various researchers in the field of eye tracking and facial gesture detection. Different systems and technologies have been employed, including real-time video processing using OpenCV, vision-based eye tracking with multiple cameras, and machine learning approaches for fatigue detection. The studies emphasize the importance of visual attention tracking, driver safety improvement, and effective human-computer interaction. However, some challenges, such as lighting conditions and performance in RGB image formats, have been identified. The proposed system using the dlib library aims to address these challenges and enable facial gesture-based cursor control and interaction.

III. TOOLS AND TECHNOLOGY

OpenCV (cv2): OpenCV is a popular computer vision library used for image and video processing. It is used for capturing video frames from the webcam, performing color space conversions, and drawing circles on the frame to visualize eye landmarks.

Mediapipe: Mediapipe is an open-source framework provided by Google that offers a collection of pre-built, customizable machine learning models for various media processing tasks. The FaceMesh model from Mediapipe is used to detect and track facial landmarks.

PyAutoGUI: PyAutoGUI is a cross-platform library used for programmatically controlling the mouse and keyboard. It provides functions to move the mouse cursor, perform clicks, and other mouse-related actions. PyAutoGUI is used to control the mouse cursor movement and perform clicks based on eye movements.

SpeechRecognition (sr): SpeechRecognition is a library that provides an interface to various speech recognition engines, including Google Speech Recognition.

pyttsx3: pyttsx3 is a cross-platform text-to-speech library. It provides a simple interface to convert text into speech. Ipyttsx3 is used to output spoken messages based on certain events, such as performing mouse clicks or scrolling.

listener: The listener variable is an instance of the Recognizer class from the SpeechRecognition library. It provides functionality for recognizing speech input from various sources, such as the microphone or audio files.

engine: The engine variable is an instance of the pyttsx3.init() function, which initializes the text-to-speech synthesis engine. It allows converting text into spoken words using the pyttsx3 library.

screen_w, screen_h: These variables store the width and height of the screen obtained using the pyautogui.size() function. They represent the dimensions of the screen on which the mouse cursor will be controlled.

cam: The cam variable is an instance of the VideoCapture class from the OpenCV library. It represents the webcam or video capture device used to capture video frames.

face_mesh: The face_mesh variable is an instance of the FaceMesh class from the Mediapipe library. It represents the FaceMesh model, which is responsible for detecting and tracking facial landmarks in the video frames.

IV. METHODOLOGY

Initializing Speech Recognition and Text-to-Speech Engines:

The code begins by initializing the listener object from the speech_recognition library as a recognizer. This object will be used to listen for and recognize speech input. The engine object from the pyttsx3 library is initialized as a text-to-speech engine. It will be used to convert text into audible speech.

Obtaining Screen Size:
The pyautogui.size() function is used to retrieve the screen size (width and height) of the computer screen. These dimensions are stored in the screen_w and screen_h variables, respectively.

Opening Webcam Capture:
The cv2.VideoCapture(0) function initializes the webcam capture. The argument 0 indicates that the default webcam should be used. The captured video frames will be processed in the subsequent steps.

Initializing FaceMesh Model:
The mp.solutions.face_mesh.FaceMesh() function initializes the FaceMesh model from the mediapipe library. This model is responsible for detecting facial landmarks. The initialized face_mesh object will be used to process video frames and extract facial landmark information.

Defining the talk() Function:
The talk(text) function utilizes the engine object to convert the provided text into audible speech. engine.say(text) sets the text to be spoken by the engine.
engine.runAndWait() initiates the speech synthesis and waits for the speech to be spoken before continuing.

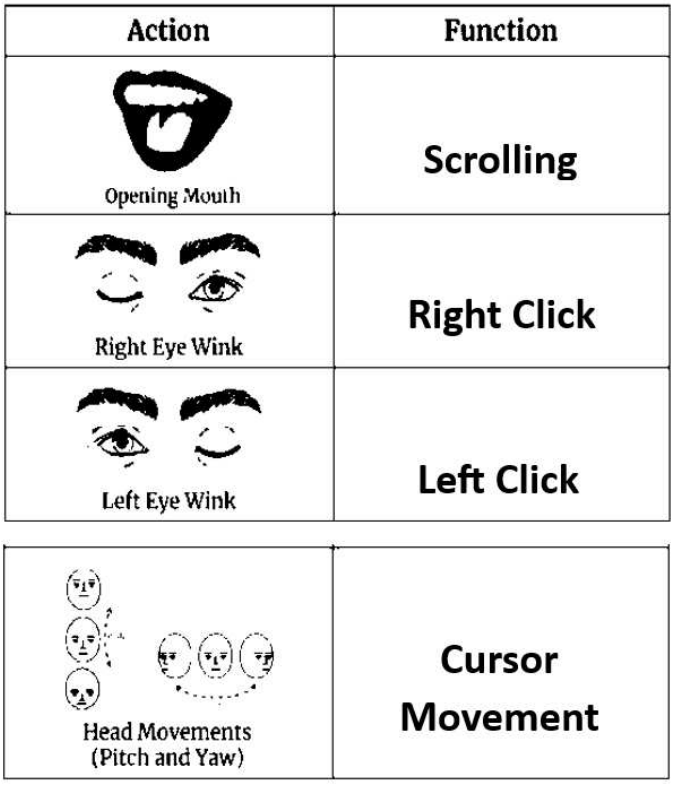


Fig 1. Actions and Functions

Main Loop:
The code enters a continuous loop using while True to continuously process video frames from the webcam.

Reading and Preprocessing Frames:
cam.read() retrieves the next video frame from the webcam capture. The returned values are stored in the variables _, frame. The captured frame is flipped horizontally using cv2.flip(frame, 1). This is done to create a mirror effect so that the user's movements appear as expected. cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) converts the color space of the frame from BGR to RGB. FaceMesh requires RGB input.

FaceMesh Processing:
face_mesh.process(rgb_frame) processes the RGB frame using the FaceMesh model. The resulting output contains information about the detected facial landmarks, such as their coordinates. The landmark_points variable stores the list of detected landmarks for all faces present in the frame.

Eye-Controlled Mouse Actions:

If there are landmark points detected (if landmark_points), the code proceeds to extract specific landmarks and perform mouse actions based on their movements. The coordinates of the landmarks are calculated based on the frame size (frame_w and frame_h). The landmarks for the left eye (left) are used to perform left-click actions. If the vertical distance between the two landmarks is below a certain threshold, a left-click is triggered using pyautogui.click(), and the text "left click" is spoken using the talk() function. Similarly, the landmarks for the right eye (right) are used to perform right-click actions. If the vertical distance between the two landmarks is below a threshold, a right-click is triggered using pyautogui.click(button='right'), and the text "Right Click" is spoken. Another set of landmarks (right1) is used to detect scrolling actions. If the vertical distance between these landmarks exceeds a certain threshold, the spacebar is pressed using `pyautogui.press('space')

Displaying Landmarks and Mouse Actions:
The code uses OpenCV (cv2.circle()) to draw circles on the frame at the coordinates of the detected landmarks. These circles serve as visual indicators for the detected facial landmarks related to mouse control. The screen_x and screen_y variables store the screen coordinates corresponding to the landmark positions. The pyautogui.moveTo(screen_x, screen_y) function moves the mouse cursor to the specified screen coordinates. The code also includes print statements to display messages in the console, indicating the occurrence of mouse actions.

Scrolling Action:
Another set of landmarks (right1) is used to detect scrolling actions. If the vertical distance between the two landmarks exceeds a certain threshold, the spacebar is pressed using pyautogui.press('space'). Additionally, the text "Doing scrolling !!" is spoken using the talk() function, and the message "space" is printed in the console.

Displaying the Processed Frame:
The processed frame, which includes the visual indicators and overlays for the detected landmarks, is displayed using cv2.imshow('Eye Controlled mouse', frame). The cv2.waitKey(1) function waits for 1 millisecond for a key event. This allows the frame to be displayed and for user interaction with the program.

Continuous Loop:
The code remains in the continuous loop, continuously reading frames from the webcam, processing them, and performing mouse actions based on the detected facial landmarks. The code essentially uses the FaceMesh model to track specific landmarks on the face, primarily focusing on the landmarks related to the left and right eyes. Based on the movements of these landmarks, the code triggers corresponding mouse actions, such as left-click, right-click, and scrolling. Additionally, it utilizes text-to-speech functionality to provide audible feedback for certain actions.

Summary: -

Initialization:
The code initializes the speech recognition (sr.Recognizer()) and text-to-speech (pyttsx3.init()) components. It also obtains the screen dimensions using pyautogui.size().

Video Capture and Face Mesh:
The code captures video frames from the webcam using cv2.VideoCapture(0). It uses the Mediapipe library and the FaceMesh model (mp.solutions.face_mesh.FaceMesh()) for facial landmark detection. The captured frames are flipped and converted to RGB color space for processing.

Mouse Control and Speech Feedback:
The code processes the facial landmarks and performs mouse actions based on the detected movements. The left eye's vertical movement is used for left-click action, and the right eye's vertical movement is used for right-click action. Scrolling action is performed based on the movement of specific landmarks. The talk () function is used to provide audible feedback for certain actions.

Displaying Processed Frame:
The processed frame, including visual indicators for the detected landmarks, is displayed using cv2.imshow().
The cv2.waitKey(1) function waits for a key event to continue the loop. The code continuously tracks the facial landmarks, detects eye movements, and performs corresponding mouse actions. It provides visual feedback on the frame and audible feedback using text-to-speech.

V. RESULTS AND DISCUSSION

User Variability: The accuracy and effectiveness of the system can vary among different users due to factors such as variations in facial structure, eye shapes, and individual characteristics. The system may need to be calibrated or personalized for each user to achieve optimal performance.

System Calibration: Proper calibration of the system is crucial for accurate tracking and control. Calibration involves establishing a mapping between facial landmarks and corresponding mouse movements. Inaccurate calibration can result in imprecise cursor movements or incorrect actions.

Real-time Performance: The responsiveness and real-time performance of the system are important for a seamless user experience. Delays or lags between facial movements and corresponding mouse actions can negatively impact usability and accuracy.

Robustness to Facial Expressions and Gestures: The system should be designed to handle various facial expressions and gestures without falsely triggering mouse actions. It should be able to distinguish between intentional eye movements and unintentional movements caused by expressions or gestures.

Noise Reduction and Filtering: To improve accuracy, techniques such as noise reduction and filtering can be applied to the captured video frames or extracted landmarks. These techniques help minimize the effects of noise, jitter, or sudden movements that could affect the reliability of the system.

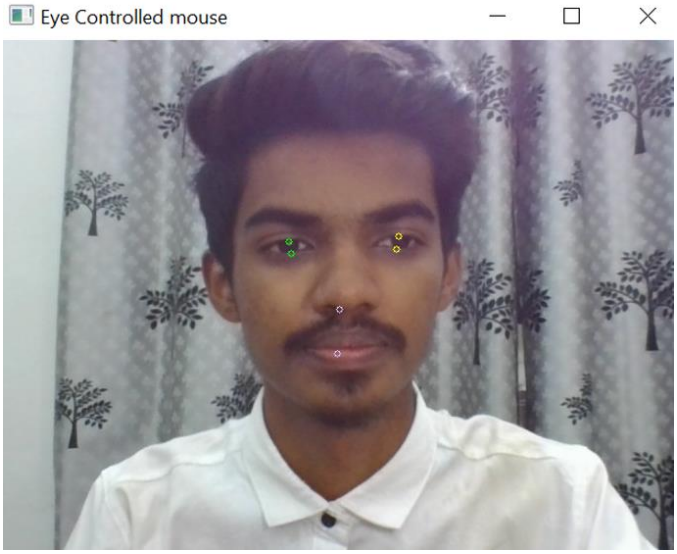


Fig 2. Landmarks Detection for Eyes and Lips

User Training and Adaptation: Providing user training or adaptation features can help users become familiar with the system and improve their interaction skills. Users may need

time to adjust to the control mechanism and learn the specific gestures or movements required for accurate control.

Limitations and Constraints: It's important to communicate any limitations or constraints of the system to users. For example, the system may have difficulty tracking fast eye movements or may not perform optimally in certain lighting conditions or camera setups.

User Feedback and Iterative Improvement: Gathering feedback from users and incorporating their suggestions and experiences can contribute to iterative improvement of the system. User feedback can help identify areas for enhancement, usability issues, and potential bugs that may affect the overall performance.

Environmental Conditions: The environmental conditions in which the system is used can affect its performance. Factors such as lighting conditions, background noise, and distractions in the user's surroundings can impact the accuracy of eye tracking and gesture recognition. It is important to consider and account for these environmental factors to ensure reliable and consistent performance.

Eye Health and Fatigue: The condition of the user's eyes and their level of fatigue can influence the accuracy and reliability of eye tracking. Users with certain eye conditions or fatigue may experience difficulties in maintaining steady eye movements, resulting in less precise control. The system should be designed to accommodate different eye conditions and consider user comfort to mitigate these challenges.



Fig 3. Implementation of Eye Cursor Control and Auditory Feedback

System Constraints and Compatibility: The system may have certain hardware or software requirements that need to be met for optimal performance. For example, specific camera specifications, processing power, or operating system compatibility may be necessary. Adhering to these constraints and ensuring compatibility can contribute to the system's effectiveness.

User Interaction Preferences: Different users may have varying preferences and comfort levels regarding the control mechanism. Some users may prefer more sensitive eye tracking, while others may prefer a more relaxed or less sensitive control. Providing options for customization and adjusting the system's sensitivity can enhance user satisfaction and usability.

System Accuracy and Calibration: The accuracy of eye tracking and gesture recognition algorithms plays a crucial role in the system's performance. The system should undergo rigorous testing and calibration to ensure precise and reliable tracking of eye movements and gestures. Regular recalibration may be necessary to maintain accuracy over time.

System Complexity and Learning Curve: The complexity of the system and the learning curve required for users to become proficient in its use can impact the results. A system that is intuitive, easy to understand, and requires minimal training can lead to quicker adoption and improved user performance.

Integration with Other Applications: The integration of the eye-controlled mouse system with other applications or software can influence its overall functionality and usefulness. Compatibility with popular operating systems, productivity tools, or gaming applications can expand the system's capabilities and provide a more versatile user experience.

VI. CONCLUSION

The research showcases the implementation of an innovative eye-controlled mouse system that integrates computer vision, gesture recognition, and speech synthesis technologies. It allows users to interact with a computer interface hands-free. The system utilizes various libraries, including OpenCV, Mediapipe, PyAutoGUI, SpeechRecognition, and pyttsx3. By utilizing the capabilities of Mediapipe's face mesh detection, the system accurately tracks facial landmarks and eye movements. It can recognize gestures such as left-clicking, right-clicking, and scrolling based on the identified facial landmarks. PyAutoGUI enables seamless interaction with the computer's graphical interface, enabling precise mouse clicks and movements corresponding to the recognized gestures. Additionally, the integration of speech recognition and synthesis technologies enhances the user experience. The SpeechRecognition library converts spoken input into text, enabling voice commands and control. The pyttsx3 library is utilized for text-to-speech synthesis, allowing the system to provide auditory feedback and information to the user. The implemented eye-controlled mouse system serves as a proof-of-concept, demonstrating the potential for hands-free computer interaction. While the code represents a foundational implementation, there is room for further refinement in terms of accuracy, performance, and user experience. It is important to consider various factors that can impact the system's

performance, including lighting conditions, user eye health, system calibration, and individual user preferences. Thorough testing, optimization, and addressing these factors are necessary to ensure reliable and precise control of the system. The eye-controlled mouse system offers an innovative and accessible approach to computer interaction, particularly beneficial for individuals with physical disabilities or limitations. Further research, development, and user feedback will contribute to refining the system, improving its accuracy and performance, and customizing it to cater to the specific needs and preferences of users.

ACKNOWLEDGEMENT

I would like to thank Prof. Gayatri Jagnade mam and Dr. Sunil Sable sir from Vishwakarma Institute of Technology, Pune for constant support and help.

REFERENCES

[1] Morimoto, Carlos, Dave Koons, Arnon Amir, and Myron Flickner. "Real-time detection of eyes and faces."In Workshop on Perceptual User Interfaces, pp. 117-120,1998.

[2] Viola, Paul, and Michael Jones. "Fast and robust classification using asymmetric adaboost and a detector cascade."In Advances in neural information processing systems, pp.1311-1318,2002.

[3] Ruddarraju, Ravikrishna, Antonio Haro , Kris Nagel, Quan T. Tran, Irfan A. Essa, Gregory Abowd, and Elizabeth D. Mynatt."Perceptual user interfaces using vision-based eye tracking." In Proceedings of the 5th international conference on Multimodal interfaces, vol.no.5, pp.227-233, 2003.

[4] Anuar, Ammar, et al. "OpenCV based real-time video processing using android smartphone. "International Journal of Computer Technology and Electronics Engineering ,vol.no.3, pp: 1-6, 2011.

[5] Wankhede, Shrunkhala Satish, S. Chhabria, and R.V. Dharaskar. "Controlling mouse cursor using eye movement." International Journal of Application or Innovation in Engineering & Management 36, pp. 17,2013.

[6] He, J., Roberson, S., Fields, B., Peng, J., Cielocha, S. and Coltea, J. “Fatigue detection using smartphones”. Journal of Ergonomics, vol.no.3(03), pp.1-7, 2013.

[7] Dey, Sanjay, et al. "Real Time Driver Fatigue Detection Based on Facial Behaviour along with Machine Learning Approaches”. IEEE International Conference on Signal Processing, Information, Communication & Systems (SPICSCON), pp. 135-140, 2019.

[8] Dr. Kavitha. R, Nachammai. N, Ranjani. R, Shifali. J, “Speech- Based Voice Recognition System for Natural Language Processing”, International Journal of Computer Science and Information Technology, Vol. 5 (4).

[9] Das, Prerana & Acharjee, Kakali & Das, Pranab & Prasad, Vijay, “voice recognition system: speech-totext”, Journal of Applied and Fundamental Sciences. 1. 2395-5562, 2015.

[10] Bhupinder Singh, Neha Kapur, Puneet Kaur “Speech Recognition with Hidden Markov Model: A Review” International Journal of Advanced Research in Computer and Software Engineering, Vol. 2, Issue 3, March 2012.

[11] Yashpalsing Chavhan, M. L. Dhore, Pallavi Yesaware, “Speech Emotion Recognition Using Support Vector Machine”, International Journal of Computer Applications, vol.1, pp.6- 9, February 2010.

[12] Stephen J. Wright, Dimitri Kanevsky, LiDeng, Xiaodong He, Georg Heigold and Haizhou Li, “Optimization Algorithms and Applications for Speech and Language Processing”, IEEE Transactions on Audio, Speech and Language Processing, 1558-7916, 2013.

[13] Kavita Sharma, Prateek Hakar “Speech Denoising Using Different Types of Filters” International journal of Engineering Research and Applications Vol. 2, Issue 1, Jan-Feb 2012

[14] G. Jagnade, M. Ikar, N. Chaudhari and M. Chaware, "Hand Gesture-based Virtual Mouse using Open CV," 2023 International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT), Bengaluru, India, 2023, pp. 820-825, doi: 10.1109/IDCIoT56793.2023.10053488.

[15] Mitesh Rakesh Ikar, Gayatri Jagnade, Nikita Chaudhari, " Computer Vision-based Air Canva Virtual Paint," International Journal of Trend in Research and Development, Volume 10(2)